Claude Certified Architect – Foundations (CCA-F) Prep Guide

How to Prepare (Start Here)

Follow this order. Don't skip ahead.

1. Take the official Anthropic courses → anthropic.skilljar.com Start here. These cover the exact technologies tested.

2. Read the official exam guide → Download PDF Read it fully. It contains the domain breakdown, task statements, sample questions with explanations, and hands-on exercises. This is your syllabus.

3. Do the hands-on labs → github.com/aderegil/claude-certified-architect Guided labs mapped to each domain. Don't just read — build.

4. Additional practice tests → certsafari.com/anthropic/claude-certified-architect Take the CertSafari mock exams to clear doubts and get used to the types of questions.

5. Take the official Claude mock exam → claudecertificationguide.com/mock-exam Do this last — so you know exactly how good you are before sitting the real exam.

Exam at a Glance

- **Format:** Multiple choice, single correct answer per question
- **Passing score:** 720 out of 1000
- **No guessing penalty** — always answer every question
- **Structure:** 4 scenario-based sections (randomly picked from 6 possible scenarios)

What the Exam Covers

Domain	Weight
Agentic Architecture & Orchestration	27%
Claude Code Configuration & Workflows	20%
Prompt Engineering & Structured Output	20%
Tool Design & MCP Integration	18%
Context Management & Reliability	15%

Domain 1 is more than a quarter of the exam. Prioritise agentic loops and multi-agent patterns.

The 6 Possible Exam Scenarios

You'll get 4 of these at random. Know all 6.

1. **Customer Support Agent** — agent with `get_customer`, `lookup_order`, `process_refund`, `escalate_to_human` tools; 80%+ first-contact resolution target
2. **Code Generation with Claude Code** — CLAUDE.md setup, custom commands, plan mode vs direct execution
3. **Multi-Agent Research System** — coordinator delegates to web search, document analysis, synthesis, and report generation subagents
4. **Developer Productivity** — codebase exploration using built-in tools (Read, Write, Bash, Grep, Glob) and MCP
5. **Claude Code for CI/CD** — automated code review, test generation, PR feedback in pipelines
6. **Structured Data Extraction** — JSON schema validation, edge case handling, downstream integration

---

Key Concepts to Know

Agentic loops — Check `stop_reason`: continue on `"tool_use"`, stop on `"end_turn"`. Never use arbitrary iteration caps as the primary termination signal.

Multi-agent orchestration — Coordinator manages all communication. Subagents do NOT inherit the coordinator's context — you must pass it explicitly. Spawn parallel subagents by emitting multiple `Task` calls in one response.

Programmatic enforcement vs. prompts — When a tool sequence is business-critical (e.g., verify identity before processing refund), use programmatic hooks/prerequisites. Prompt instructions have a non-zero failure rate.

Tool descriptions — The primary mechanism the LLM uses for tool selection. Always include: input formats, example queries, edge cases, and when to use this tool vs. similar ones. Minimal descriptions cause misrouting.

`tool_choice` options

- `"auto"` → model may or may not call a tool
- `"any"` → model must call a tool (guarantees structured output)
- `{"type":"tool","name":"X"}` → forces a specific tool

MCP server scoping — `.mcp.json` for project-level (shared, version-controlled); `~/.claude.json` for personal/experimental. Use environment variable expansion for credentials — never commit secrets.

CLAUDE.md hierarchy — user-level (`~/.claude/CLAUDE.md`) → project-level (`.claude/CLAUDE.md`) → directory-level. Use `.claude/rules/` with YAML glob patterns for path-specific conventions. Use `context: fork` in skills to isolate verbose output from the main session.

Plan mode vs. direct execution — Plan mode for architectural decisions, large-scale multi-file changes, multiple valid approaches. Direct execution for simple, well-scoped, single-file changes.

CI/CD — Use `-p` flag for non-interactive mode. Use `--output-format json` + `--json-schema` for structured output. Always use an independent Claude session for code review (not the same session that generated the code).

Message Batches API — 50% cost savings, up to 24h processing, no latency SLA. Use for overnight/weekly batch jobs. Never use for blocking pre-merge checks.

Escalation triggers — Escalate when: customer explicitly asks for a human, policy has a gap, or no progress can be made. Do not escalate based on sentiment or self-reported confidence alone.

Context management — Extract transactional facts (amounts, dates, order numbers) into a persistent block outside summarised history. Trim verbose tool outputs before they accumulate. Put key findings at the start of long inputs to counter the "lost in the middle" effect.

Few-shot prompting — Most effective for consistent formatted output and reducing false positives. Provide 2-4 targeted examples showing reasoning for ambiguous scenarios.

Structured output — Use `tool_use` + JSON schema to eliminate syntax errors. Mark fields as `nullable`/optional when source documents may not contain the information — prevents hallucination.

What Is NOT on the Exam

Save your time — these topics are explicitly out of scope:

- Fine-tuning or training models
- API authentication, billing, or rate limits
- Deploying or hosting MCP servers (infra, networking, containers)
- Claude's internal architecture or training methodology
- Embedding models or vector databases
- Computer use, vision, or image analysis
- Streaming API or server-sent events
- Specific cloud provider configurations (AWS, GCP, Azure)
- Prompt caching implementation details
- Token counting or tokenization specifics